Lecture 20: Representing Instructions

CMPS 221 – Computer Organization and Design

Adapted by Izzat El Hajj from textbook slides for Computer Organization and Design, by David Patterson and John Hennessy, Morgan Kaufmann.

Assembly Language

```
swap (int v[], int k) {
    int temp;
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}
```

High-level Language Program

(what the programmer writes)

Compiler

```
swap:  sll $2, $5, 2
       add $2, $4, $2
       lw  $15, 0($2)
       lw  $16, 4($2)
       sw  $16, 0($2)
       sw  $15, 4($2)
       jr  $31
```

Assembly Language Program

(what the hardware understands)

Assembler

```
000000 00000 00101 0001000010000000
000000 00100 00010 0001000000100000
                ...
```

Machine (Object) Code

(what the hardware *really* understands)

Instructions are Data

- Instructions are data stored in memory
  - To execute a program, a processor fetches the program's instructions from memory
  - The **Program Counter (PC)** is a special register (not part of the register file) that points to the instruction to be fetched from memory

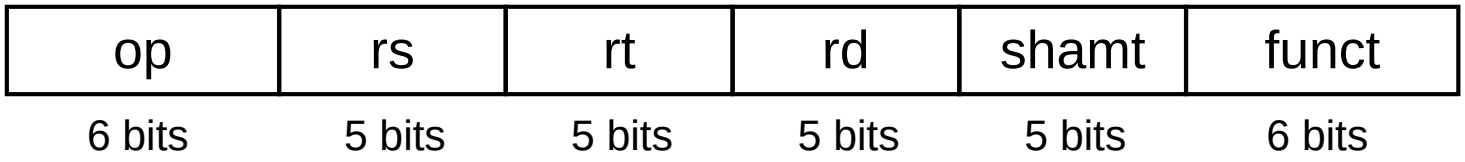
- Need a way to encode instructions in binary
  - Binary-encoded instructions called **machine code**

Representing Instructions

- MIPS instructions
  - Every instruction is encoded as a 32-bit word
  - Three formats for encoding instructions
    - R-format, I-format, J-format

- In contrast, some architectures have variable length instructions
  - Advantages: space efficiency, flexibility
  - Disadvantages: needs more complex hardware

MIPS R-format Instructions



- Instruction fields
  - op: operation code (opcode)
  - rs: first source register number
  - rt: second source register number
  - rd: destination register number
  - shamt: shift amount
    - How many positions to shift (shift instructions only)
  - funct: function code (extends opcode)

R-format Example

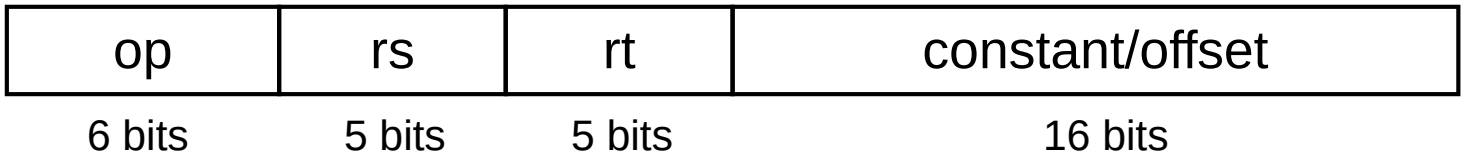
op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

```
add $t0, $s1, $s2
```

add	$s1	$s2	$t0	0	add
0	17	18	8	0	32
000000	10001	10010	01000	00000	100000

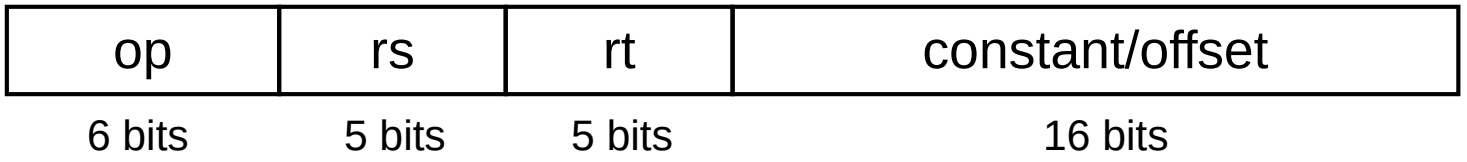
$$00000010001100100100000000100000_2 = 02324020_{16}$$

MIPS I-format Instructions



- Immediate arithmetic and load/store instructions
  - rs: first source register number
  - rt: second source register number or destination register number
  - Constant/offset: -2^{15} to $+2^{15} - 1$
    - Now we know why immediates are limited to 16 bits

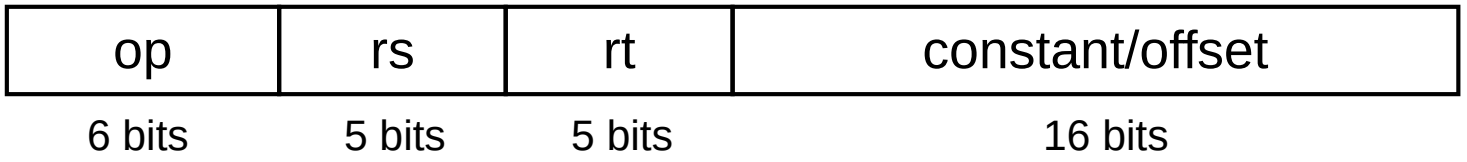
MIPS I-format Example




```
addi $s1, $s2, 100
```

addi	$s2	$s1	100
8	18	17	100
001000	10010	10001	0000000001100100

MIPS I-format Example




```
lw $s1, 100($s2)
```

lw	$s2	$s1	100
35	18	17	100
100011	10010	10001	0000000001100100

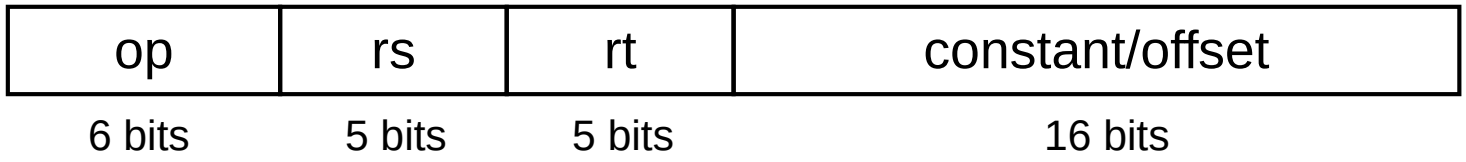
MIPS I-format Example

op	rs	rt	constant/offset
6 bits	5 bits	5 bits	16 bits

```
sw $s1, 100($s2)
```

sw	$s2	$s1	100
43	18	17	100
101011	10010	10001	0000000001100100

Branch Addressing



- Branch instructions take two registers and a target instruction address
- How to represent a 32-bit target address using just 16 bits?
  - Use **PC-Relative Addressing** to encode target address as an offset from the current instruction
    - Target address = (PC + 4) + offset × 4
    - Most branch targets are nearby (forward/backward)
    - Instruction addresses are multiples of 4 because instructions are 32-bit words

Branch Addressing Example

```
0x14080000  Loop: sll  $t1, $s3, 2
0x14080004        add  $t1, $t1, $s6
0x14080008        lw   $t0, 0($t1)
0x1408000c        bne  $t0, $s5, Exit
0x14080010        addi $s3, $s3, 1
0x14080014        j    Loop
0x14080018  Exit: ...
```

Target address = (**PC** + 4) + offset × 4

0x14080018 = (**0x1408000c** + 4) + offset × 4

offset = 2

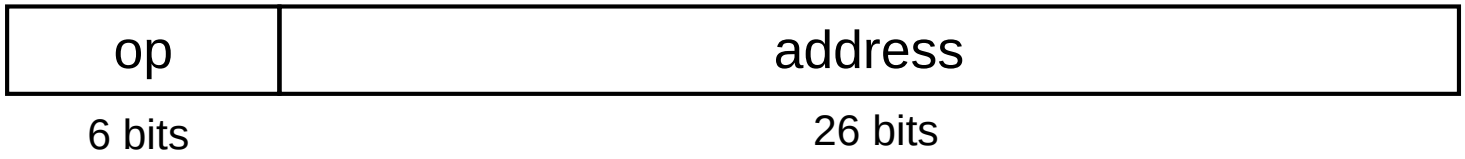
- Represent: `bne $t0, $s5, Exit`

I-format

op	rs	rt	constant or offset
bne	$t0	$s5	Exit
5	8	21	2

Jump Addressing

- Jump (`j` and `jal`) targets could be anywhere in text segment
  - Use **J-format** to encode full address in instruction



- How to represent a 32-bit target address using just 26 bits?
  - Use **Pseudo-Direct Jump Addressing**
    - Target address = $PC_{31\text{-}28}$: (address × 4)
    - Instruction addresses are multiples of 4 because instructions are 32-bit words (so lower two bits are always 0)
    - Get remaining upper 4 bits from the program counter

Jump Addressing Example

```
0x14080000  Loop: sll  $t1, $s3, 2
0x14080004        add  $t1, $t1, $s6
0x14080008        lw   $t0, 0($t1)
0x1408000c        bne  $t0, $s5, Exit
0x14080010        addi $s3, $s3, 1
0x14080014        j    Loop
0x14080018  Exit: ...
```

Target address = **$PC_{31\text{-}28}$** : (address × 4)
0x14080000 = **0x1** : (address × 4)
0x4080000 = address × 4
address = 0x1020000

- Represent: `j` **Loop**

J-format

op	address
j	Loop
2	0x1020000

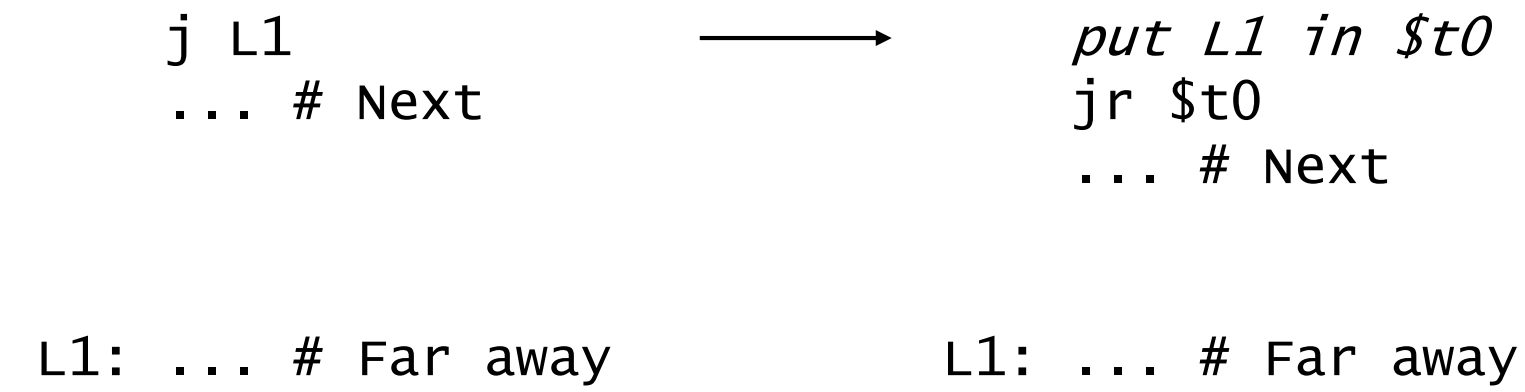
Branching Far Away

- If branch target is too far to encode with 16-bit offset, rewrite code to use `j`
- Example:

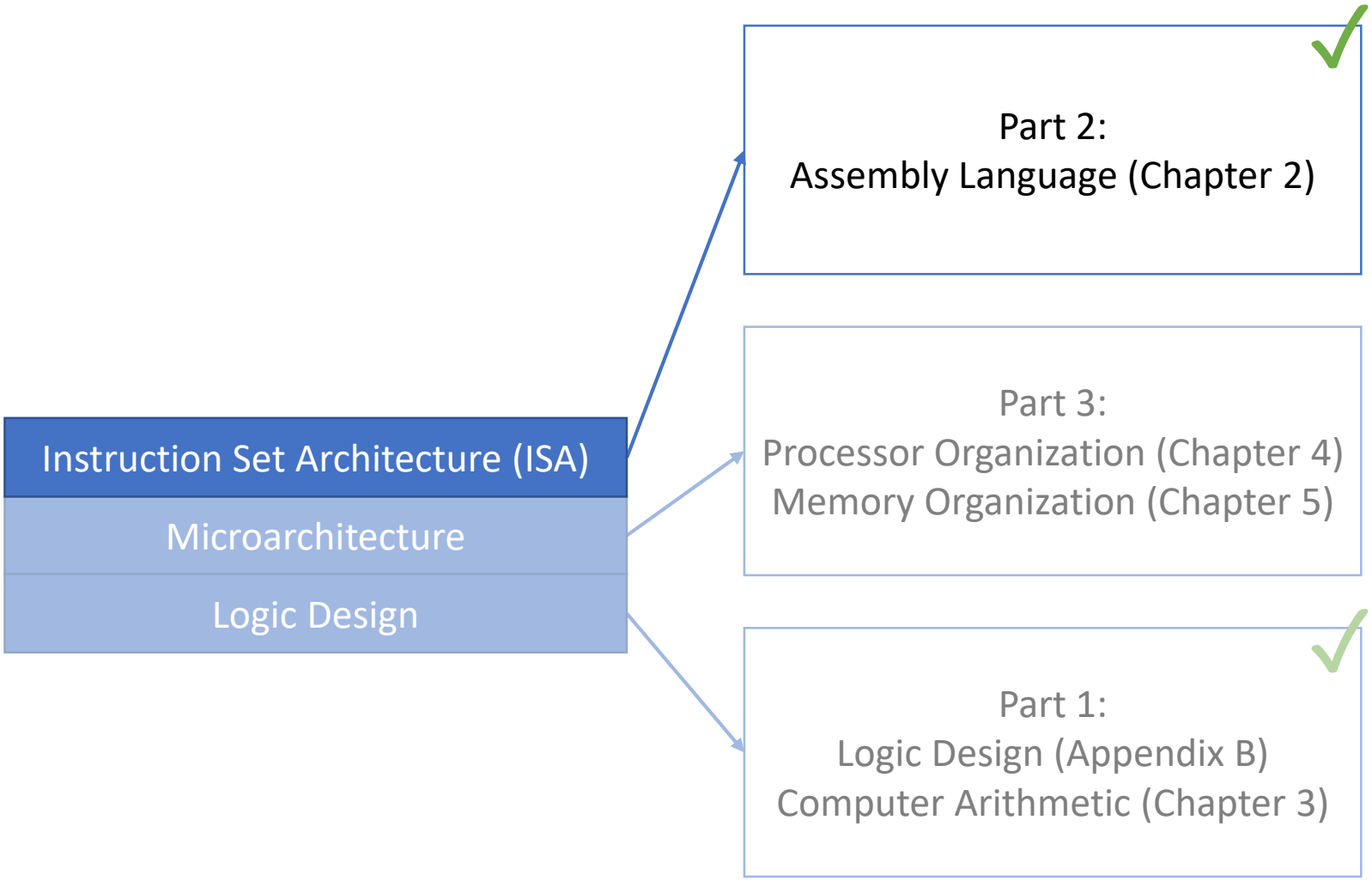
```
      beq $s0,$s1, L1   ⟶      bne $s0,$s1, L2
      ... # Next                j L1
                          L2: ... # Next


 L1: ... # Far away       L1: ... # Far away
```

Jumping Far Away

- If jump target is too far to encode with 26-bit address, rewrite code to use `jr`
- Example:



End of Part 2


Instruction Set Architecture (ISA)
Microarchitecture
Logic Design
Part 2:
Assembly Language (Chapter 2)
Part 3:
Processor Organization (Chapter 4)
Memory Organization (Chapter 5)
Part 1:
Logic Design (Appendix B)
Computer Arithmetic (Chapter 3)

Textbook Sections

- The content in these slides corresponds to:
  - Textbook:
    - *Computer Organization and Design, 5th Edition by David Patterson and John Hennessy, Morgan Kaufmann, 2014.*
  - Sections:
    - 2.5, 2.10